In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import io
```

In [2]:
```
from google.colab import files

uploaded = files.upload()

filename = next(iter(uploaded))
df = pd.read_csv(io.BytesIO(uploaded[filename]))

print("Dataset loaded successfully!")
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving dataset_socialmedia_engagement_univ_vienna.csv to dataset_socialmedia_engagem
ent_univ_vienna.csv
Dataset loaded successfully!
```

In [3]:
```
df.head(5)
```

Out[3]:

	post_id	platform	post_type	post_time	likes	comments	shares	post_day	sentimen
0	1	Facebook	image	8/17/2023 14:45	2121	474	628	Thursday	
1	2	Facebook	carousel	5/14/2023 0:45	3660	432	694	Sunday	
2	3	Instagram	poll	2/21/2023 16:15	4955	408	688	Tuesday	n
3	4	Twitter	image	11/16/2023 0:45	1183	90	187	Thursday	n
4	5	Twitter	video	5/23/2023 0:30	3499	247	286	Tuesday	

In [4]:
```
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])

df.info()
```

```
Rows: 100
Columns: 9
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype 
---  ------           --------------  ----- 
 0   post_id          100 non-null    int64 
 1   platform         100 non-null    object
 2   post_type        100 non-null    object
 3   post_time        100 non-null    object
 4   likes            100 non-null    int64 
 5   comments         100 non-null    int64 
 6   shares           100 non-null    int64 
 7   post_day         100 non-null    object
 8   sentiment_score  100 non-null    object
dtypes: int64(4), object(5)
memory usage: 7.2+ KB
```

Data Cleaning

The dataset was checked for missing values and duplicate records to assess data quality. No missing values or duplicate rows were identified in the filtered dataset. Therefore, no imputation or removal of records was necessary.

In [5]:
```
print("Missing values:\n", df.isnull().sum())
print("\nDuplicate rows:", df.duplicated().sum())
```

```
Missing values:
 post_id            0
platform           0
post_type          0
post_time          0
likes              0
comments           0
shares             0
post_day           0
sentiment_score    0
dtype: int64

Duplicate rows: 0
```

Data Filtering

The original dataset was filtered to include only Instagram and Facebook posts, as these were the two platforms selected for this study. This reduced the dataset to 68 observations. Filtering ensured that the analysis remained focused on the platforms identified in the research scope.

In [6]:
```
#Extracting only Instagram and Facebook data from the entire dataset
```

```
df_filtered = df[df['platform'].isin(['Instagram', 'Facebook'])].copy()
```

Exploratory Data Analysis (EDA)

In [7]:
```
df_filtered.describe().round(1)
```

Out[7]:

	post_id	likes	comments	shares
count	68.0	68.0	68.0	68.0
mean	54.7	2841.1	240.2	501.3
std	29.0	1632.1	150.4	290.7
min	1.0	15.0	10.0	16.0
25%	30.8	1655.2	103.2	239.8
50%	59.0	3015.5	221.5	458.0
75%	79.2	4603.8	362.0	740.2
max	100.0	4992.0	500.0	993.0

In [8]:
```
df_filtered['platform'].value_counts().to_frame('Number of Posts')
```

Out[8]:

platform	Number of Posts
Instagram	36
Facebook	32

In [9]:
```
df_filtered.groupby('post_type')[['likes', 'comments', 'shares']].mean().round(1)
```

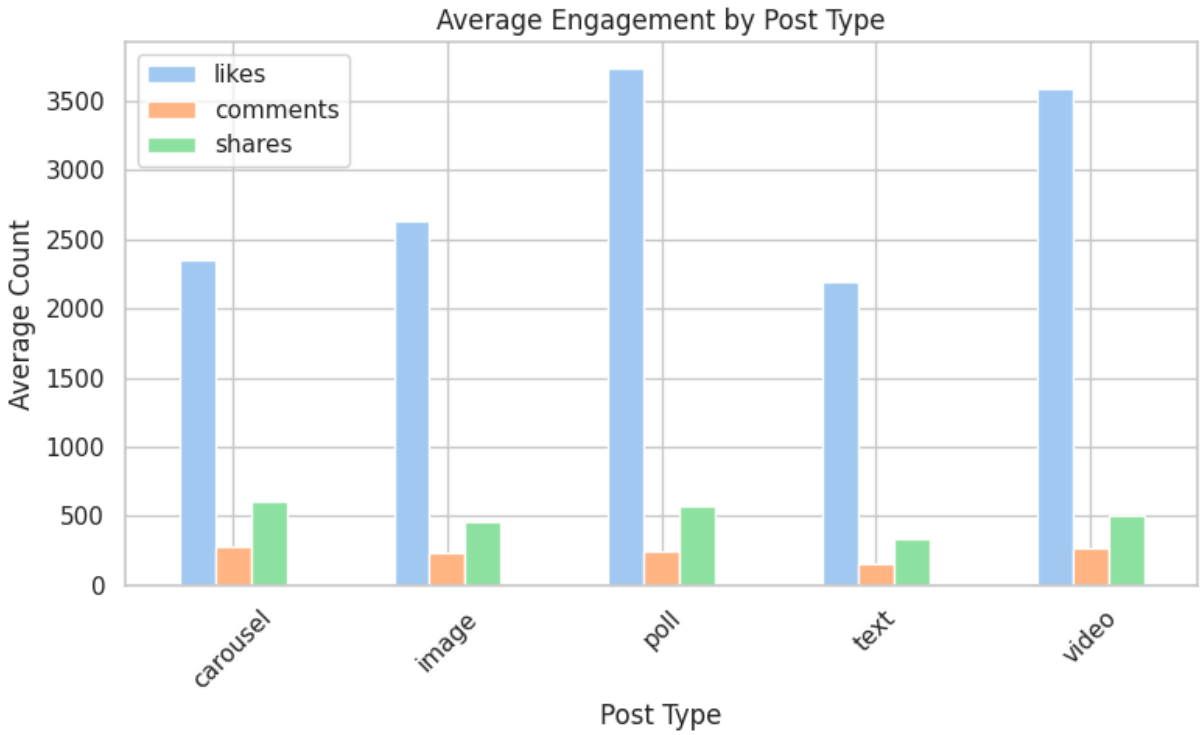
Out[9]:

post_type	likes	comments	shares
carousel	2352.7	276.9	602.9
image	2630.6	234.0	461.4
poll	3734.8	241.6	568.0
text	2197.8	158.6	336.8
video	3585.9	266.4	504.0

In [10]:
```
sns.set_theme(style="whitegrid", palette="pastel")
df_filtered.groupby('post_type')[['likes', 'comments', 'shares']].mean().plot(
    kind='bar',
```

```
    figsize=(8, 5)
)

plt.title('Average Engagement by Post Type')
plt.xlabel('Post Type')
plt.ylabel('Average Count')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



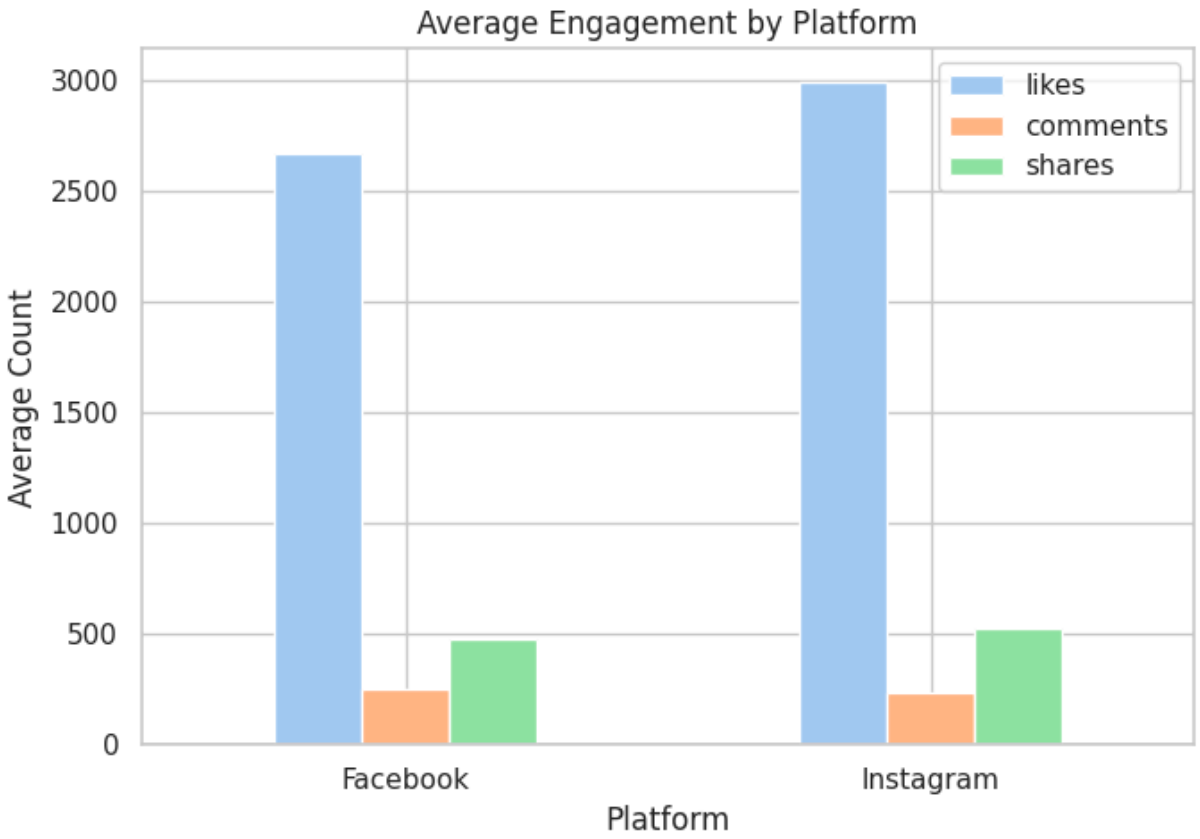
In [11]:
```
df_filtered.groupby('platform')[['likes', 'comments', 'shares']].mean().round(1)
```

Out[11]:

platform	likes	comments	shares
Facebook	2669.3	248.9	474.4
Instagram	2993.9	232.4	525.2

In [12]:
```
df_filtered.groupby('platform')[['likes', 'comments', 'shares']].mean().plot(
    kind='bar',
    figsize=(7, 5)
)

plt.title('Average Engagement by Platform')
plt.xlabel('Platform')
plt.ylabel('Average Count')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



In [13]:
```
df_filtered.groupby('post_day')[['likes', 'comments', 'shares']].mean().round(1)
```

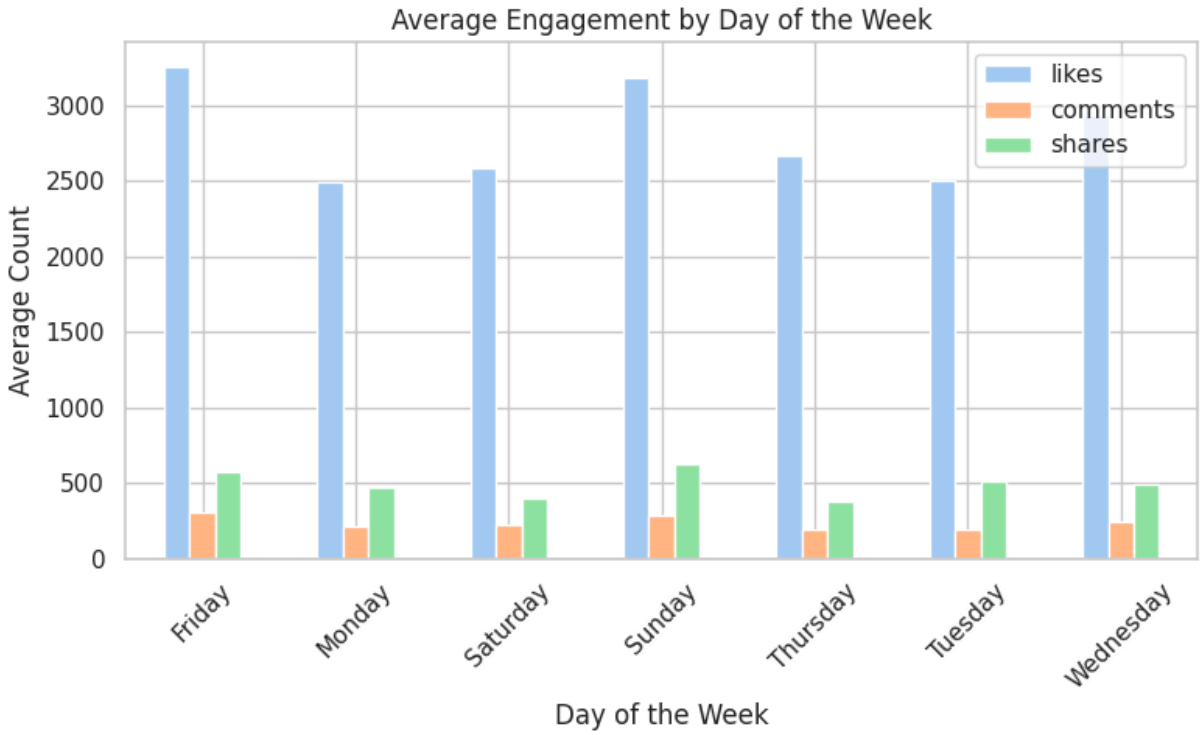
Out[13]:

post_day	likes	comments	shares
Friday	3252.3	306.9	580.3
Monday	2495.2	211.6	475.4
Saturday	2585.5	223.0	395.5
Sunday	3180.2	286.6	629.9
Thursday	2664.4	192.2	376.2
Tuesday	2505.0	195.1	515.7
Wednesday	2936.5	251.0	493.6

In [14]:
```
df_filtered.groupby('post_day')[['likes', 'comments', 'shares']].mean().plot(
    kind='bar',
    figsize=(8, 5)
)

plt.title('Average Engagement by Day of the Week')
plt.xlabel('Day of the Week')
plt.ylabel('Average Count')
plt.xticks(rotation=45)
```

```
plt.tight_layout()
plt.show()
```



Data Transformation: Posting Time

The original post_time variable was converted into a numerical post_hour variable using Python's datetime functionality. This extracted the hour of the day (0–23) when each post was published, making posting time suitable for numerical analysis.

In [15]:
```
df_filtered['post_time'].head()
```

Out[15]:

	post_time
0	8/17/2023 14:45
1	5/14/2023 0:45
2	2/21/2023 16:15
5	5/5/2023 20:00
6	2/26/2023 11:45

dtype: object

In [16]:
```
df_filtered['post_hour'] = pd.to_datetime(
    df_filtered['post_time'], format='mixed'
).dt.hour
```

In [17]:
```
df_filtered[['post_time', 'post_hour']].head()
```

Out[17]:

	post_time	post_hour
0	8/17/2023 14:45	14
1	5/14/2023 0:45	0
2	2/21/2023 16:15	16
5	5/5/2023 20:00	20
6	2/26/2023 11:45	11

Exploratory Data Analysis (EDA) continued

In [18]:
```
df_filtered.groupby('post_hour')[['likes', 'comments', 'shares']].mean().round(1)
```

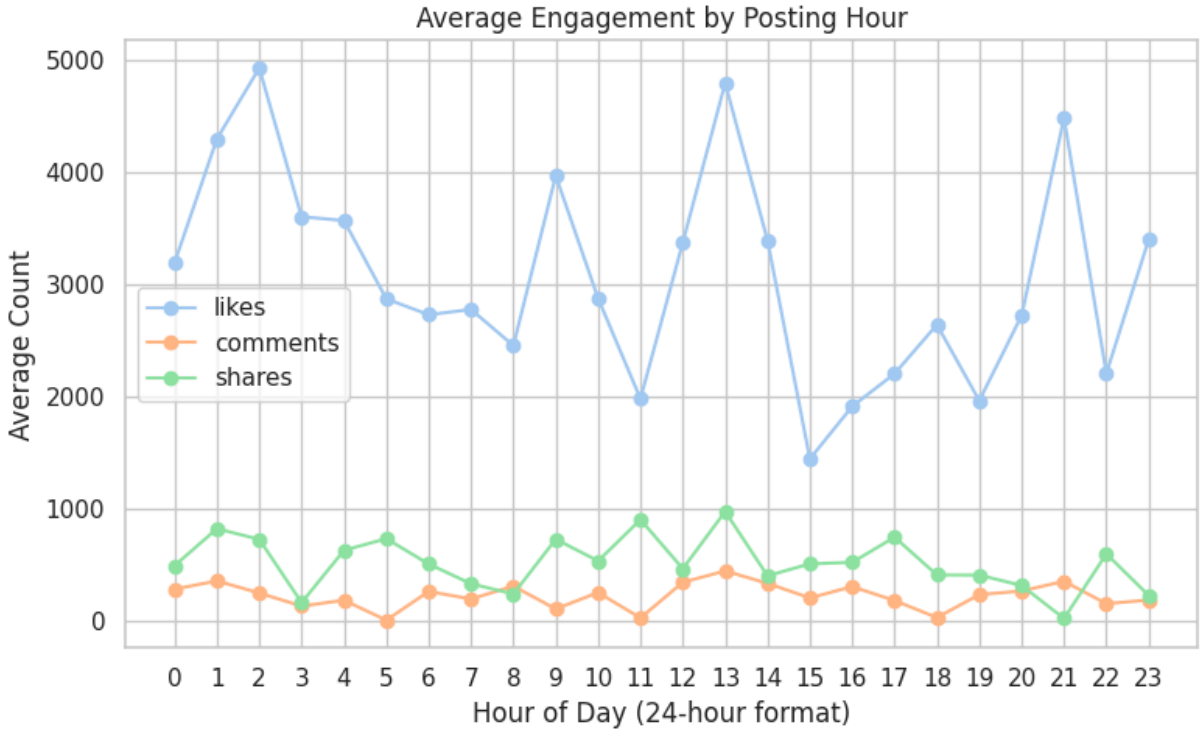
Out[18]:

post_hour	likes	comments	shares
0	3193.6	286.9	495.6
1	4297.0	360.0	825.0
2	4929.0	252.0	730.0
3	3604.0	135.5	167.0
4	3569.8	187.7	632.3
5	2872.0	10.0	736.0
6	2731.0	265.8	510.5
7	2778.0	199.0	334.5
8	2456.0	313.3	243.3
9	3970.0	110.0	731.5
10	2877.0	256.3	537.3
11	1982.0	30.0	906.0
12	3374.8	347.8	468.2
13	4795.0	449.0	978.0
14	3393.0	338.0	405.5
15	1446.7	208.3	512.0
16	1915.0	309.2	525.0
17	2206.3	184.7	749.3
18	2642.0	33.0	413.0
19	1965.4	238.0	411.9
20	2724.2	270.0	319.0
21	4483.0	357.0	25.0
22	2209.0	157.0	600.8
23	3401.0	190.0	230.0

In [19]:
```
df_filtered.groupby('post_hour')[['likes', 'comments', 'shares']].mean().plot(
    kind='line',
    marker='o',
    figsize=(8, 5)
)

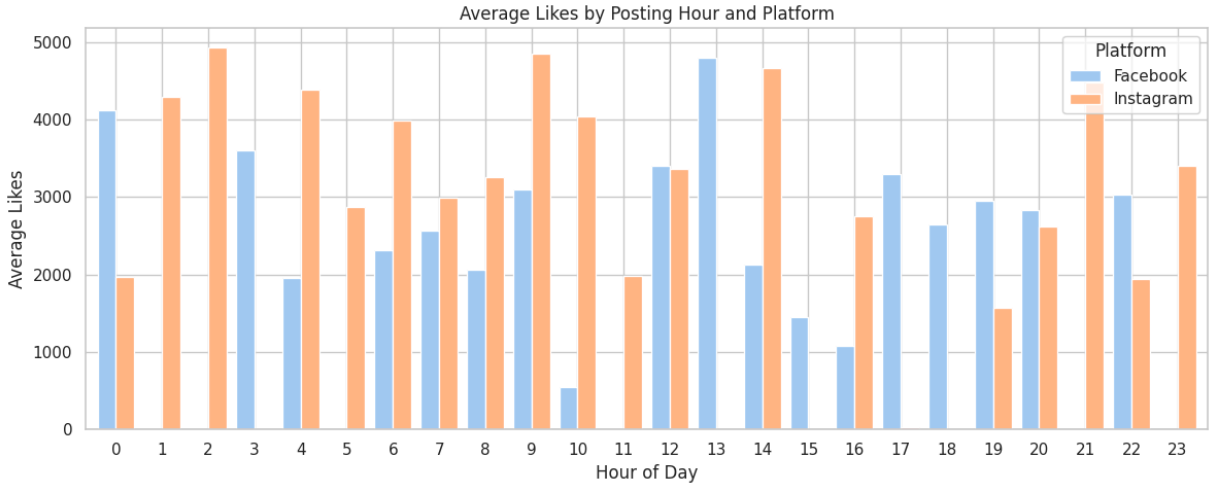
plt.title('Average Engagement by Posting Hour')
plt.xlabel('Hour of Day (24-hour format)')
```

```
plt.ylabel('Average Count')
plt.xticks(range(0, 24))
plt.tight_layout()
plt.show()
```



In [20]:
```
df_filtered.groupby(['post_hour', 'platform'])['likes'].mean().unstack().plot(
    kind='bar',
    figsize=(12, 5),
    width=0.8
)

plt.title('Average Likes by Posting Hour and Platform')
plt.xlabel('Hour of Day')
plt.ylabel('Average Likes')
plt.xticks(rotation=0)
plt.legend(title='Platform')
plt.tight_layout()
plt.show()
```



In [21]:
```
# Count posts by post type and platform
print("Posts by Type:")
print(df_filtered['post_type'].value_counts())

print("\nPosts by Platform:")
print(df_filtered['platform'].value_counts())

# Visualize distributions

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

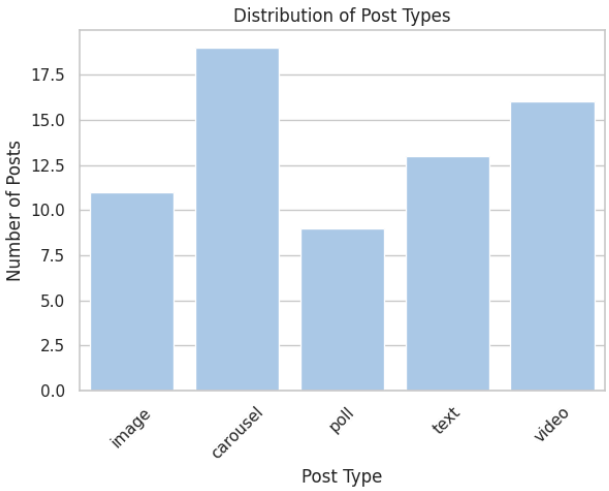
sns.countplot(data=df_filtered, x='post_type', ax=axes[0])
axes[0].set_title('Distribution of Post Types')
axes[0].set_xlabel('Post Type')
axes[0].set_ylabel('Number of Posts')
axes[0].tick_params(axis='x', rotation=45)

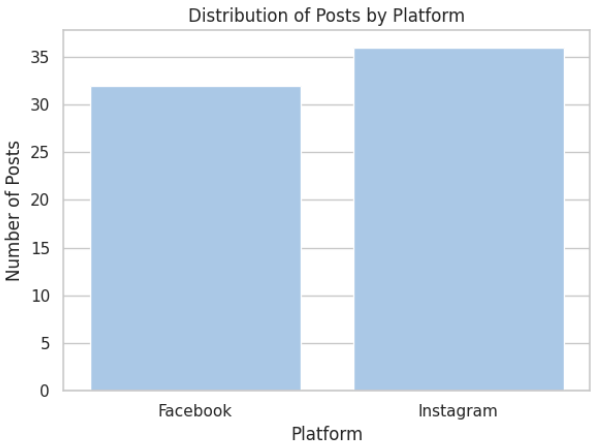
sns.countplot(data=df_filtered, x='platform', ax=axes[1])
axes[1].set_title('Distribution of Posts by Platform')
axes[1].set_xlabel('Platform')
axes[1].set_ylabel('Number of Posts')

plt.tight_layout()
plt.show()
```

```
Posts by Type:
post_type
carousel    19
video       16
text        13
image       11
poll         9
Name: count, dtype: int64

Posts by Platform:
platform
Instagram    36
Facebook     32
Name: count, dtype: int64
```





In [22]:
```
# Summary statistics for engagement
engagement_columns = ['likes', 'comments', 'shares']

print("Engagement Summary Statistics:")
display(df_filtered[engagement_columns].describe().round(2))

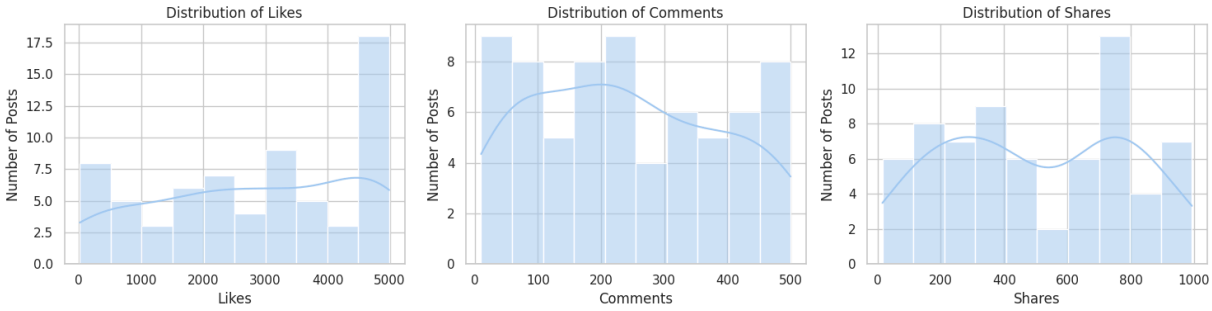
# Visualize the distribution of each engagement measure
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for i, column in enumerate(engagement_columns):
    sns.histplot(data=df_filtered, x=column, bins=10, kde=True, ax=axes[i])
    axes[i].set_title(f'Distribution of {column.title()}')
    axes[i].set_xlabel(column.title())
    axes[i].set_ylabel('Number of Posts')

plt.tight_layout()
plt.show()
```

Engagement Summary Statistics:

	likes	comments	shares
count	68.00	68.00	68.00
mean	2841.13	240.19	501.26
std	1632.13	150.38	290.68
min	15.00	10.00	16.00
25%	1655.25	103.25	239.75
50%	3015.50	221.50	458.00
75%	4603.75	362.00	740.25
max	4992.00	500.00	993.00



In [23]:
```
# Calculate average engagement by platform and post type
engagement_by_type = (
    df_filtered
    .groupby(['platform', 'post_type'])[['likes', 'comments', 'shares']]
    .mean()
    .round(2)
)

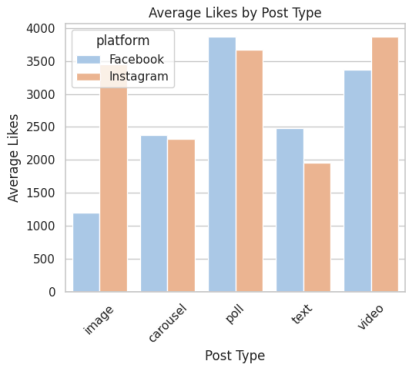
display(engagement_by_type)

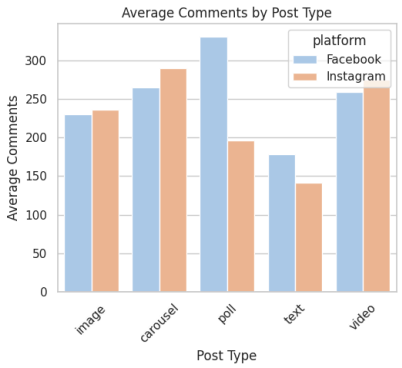
# Visualize likes, comments, and shares
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

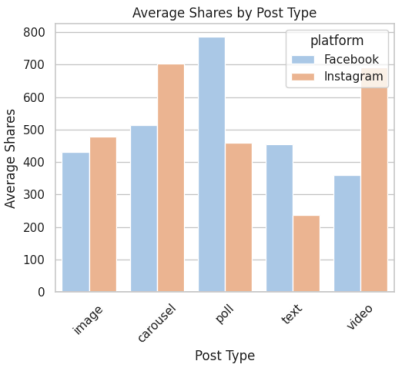
for i, column in enumerate(['likes', 'comments', 'shares']):
    sns.barplot(
        data=df_filtered,
        x='post_type',
        y=column,
        hue='platform',
        ax=axes[i],
        errorbar=None
    )
    axes[i].set_title(f'Average {column.title()} by Post Type')
    axes[i].set_xlabel('Post Type')
    axes[i].set_ylabel(f'Average {column.title()}')
    axes[i].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

platform	post_type	likes	comments	shares
Facebook	carousel	2379.70	264.90	513.40
	image	1204.50	230.00	432.00
	poll	3868.00	331.00	786.33
	text	2484.67	178.00	454.50
	video	3365.56	259.44	359.11
Instagram	carousel	2322.67	290.22	702.44
	image	3445.57	236.29	478.14
	poll	3668.17	196.83	458.83
	text	1952.00	142.00	236.00
	video	3869.14	275.29	690.29







In [24]:
```
# Check sample sizes for our main variables
print("Posts by Platform:")
display(df_filtered['platform'].value_counts().to_frame('Number of Posts'))

print("Posts by Post Type:")
display(df_filtered['post_type'].value_counts().to_frame('Number of Posts'))

print("Posts by Day of Week:")
display(df_filtered['post_day'].value_counts().to_frame('Number of Posts'))

print("Posts by Posting Hour:")
display(df_filtered['post_hour'].value_counts().sort_index().to_frame('Number of Po

# Check post type counts within each platform
print("Post Type Counts by Platform:")
display(pd.crosstab(df_filtered['post_type'], df_filtered['platform']))
```

```
Posts by Platform:
```

platform	Number of Posts
Instagram	36
Facebook	32

```
Posts by Post Type:
```

post_type	Number of Posts
carousel	19
video	16
text	13
image	11
poll	9

```
Posts by Day of Week:
```

post_day	Number of Posts
Wednesday	13
Thursday	12
Sunday	11
Tuesday	11
Friday	10
Saturday	6
Monday	5

```
Posts by Posting Hour:
```

	Number of Posts
post_hour	
0	7
1	1
2	1
3	2
4	6
5	1
6	4
7	2
8	3
9	2
10	3
11	1
12	4
13	1
14	2
15	3
16	4
17	3
18	1
19	7
20	4
21	1
22	4
23	1

```
Post Type Counts by Platform:
```

platform	Facebook	Instagram
post_type		
carousel	10	9
image	4	7
poll	3	6
text	6	7
video	9	7

In [25]:
```
# Average engagement by weekday and platform
weekday_results = (
    df_filtered
    .groupby(['post_day', 'platform'])[['likes', 'comments', 'shares']]
    .mean()
    .round(2)
)

print("Average Engagement by Weekday and Platform:")
display(weekday_results)

# Average engagement by posting hour and platform
hour_results = (
    df_filtered
    .groupby(['post_hour', 'platform'])[['likes', 'comments', 'shares']]
    .mean()
    .round(2)
)

print("Average Engagement by Posting Hour and Platform:")
display(hour_results)

# Visualize average likes by weekday and posting hour
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

sns.barplot(
    data=df_filtered,
    x='post_day',
    y='likes',
    hue='platform',
    errorbar=None,
    ax=axes[0]
)
axes[0].set_title('Average Likes by Day and Platform')
axes[0].set_xlabel('Day of Week')
axes[0].set_ylabel('Average Likes')
axes[0].tick_params(axis='x', rotation=45)

sns.barplot(
    data=df_filtered,
    x='post_hour',
```

```
    y='likes',
    hue='platform',
    errorbar=None,
    ax=axes[1]
)
axes[1].set_title('Average Likes by Posting Hour and Platform')
axes[1].set_xlabel('Posting Hour (24-hour clock)')
axes[1].set_ylabel('Average Likes')

plt.tight_layout()
plt.show()
```

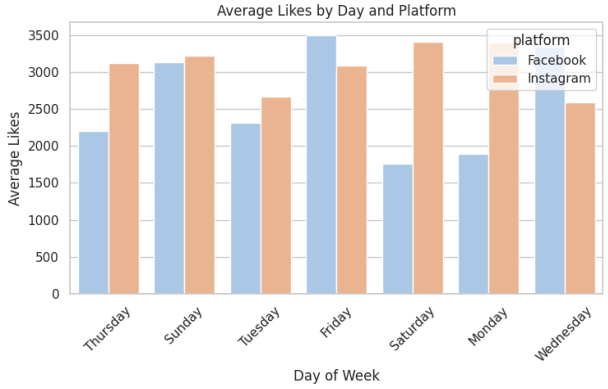
```
Average Engagement by Weekday and Platform:
```

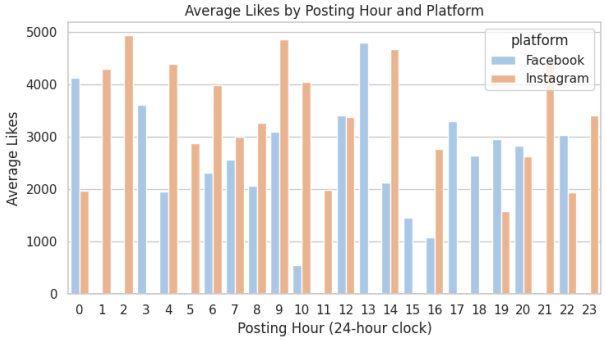
post_day	platform	likes	comments	shares
Friday	**Facebook**	3495.50	275.50	676.50
	Instagram	3090.17	327.83	516.17
Monday	**Facebook**	1892.33	202.33	505.67
	Instagram	3399.50	225.50	430.00
Saturday	**Facebook**	1759.00	243.67	383.00
	Instagram	3412.00	202.33	408.00
Sunday	**Facebook**	3133.40	404.20	625.80
	Instagram	3219.17	188.67	633.33
Thursday	**Facebook**	2206.00	212.17	325.50
	Instagram	3122.83	172.17	426.83
Tuesday	**Facebook**	2307.40	121.20	542.00
	Instagram	2669.67	256.67	493.83
Wednesday	**Facebook**	3340.17	270.83	336.00
	Instagram	2590.57	234.00	628.71

```
Average Engagement by Posting Hour and Platform:
```

post_hour	platform	likes	comments	shares
0	Facebook	4113.75	277.25	494.50
	Instagram	1966.67	299.67	497.00
1	Instagram	4297.00	360.00	825.00
2	Instagram	4929.00	252.00	730.00
3	Facebook	3604.00	135.50	167.00
4	Facebook	1949.00	161.50	509.00
	Instagram	4380.25	200.75	694.00
5	Instagram	2872.00	10.00	736.00
6	Facebook	2314.00	249.67	572.00
	Instagram	3982.00	314.00	326.00
7	Facebook	2566.00	368.00	304.00
	Instagram	2990.00	30.00	365.00
8	Facebook	2054.50	445.00	276.50
	Instagram	3259.00	50.00	177.00
9	Facebook	3094.00	86.00	965.00
	Instagram	4846.00	134.00	498.00
10	Facebook	548.00	153.00	731.00
	Instagram	4041.50	308.00	440.50
11	Instagram	1982.00	30.00	906.00
12	Facebook	3404.00	244.00	67.00
	Instagram	3365.00	382.33	602.00
13	Facebook	4795.00	449.00	978.00
14	Facebook	2121.00	474.00	628.00
	Instagram	4665.00	202.00	183.00
15	Facebook	1446.67	208.33	512.00
16	Facebook	1072.50	171.50	367.50
	Instagram	2757.50	447.00	682.50
17	Facebook	3291.50	130.00	668.50
	Instagram	36.00	294.00	911.00

		likes	comments	shares
post_hour	**platform**			
18	**Facebook**	2642.00	33.00	413.00
19	**Facebook**	2943.50	381.50	230.00
	Instagram	1574.20	180.60	484.60
20	**Facebook**	2827.00	407.00	333.00
	Instagram	2621.50	133.00	305.00
21	**Instagram**	4483.00	357.00	25.00
22	**Facebook**	3026.00	11.00	761.00
	Instagram	1936.67	205.67	547.33
23	**Instagram**	3401.00	190.00	230.00





In [26]:
```
# Overall engagement averages by platform
print("1. Average Engagement by Platform")
display(
    df_filtered.groupby('platform')[['likes', 'comments', 'shares']]
    .mean().round(2)
)

# Overall engagement averages by post type
print("\n2. Average Engagement by Post Type")
display(
    df_filtered.groupby('post_type')[['likes', 'comments', 'shares']]
    .mean().round(2)
)

# Highest-average categories for each engagement measure
print("\n3. Highest Average Engagement Categories")

for metric in ['likes', 'comments', 'shares']:
    best_type = df_filtered.groupby('post_type')[metric].mean().idxmax()
    best_platform = df_filtered.groupby('platform')[metric].mean().idxmax()

    print(f"\n{metric.title()}:")
```

```
    print(f"  Highest-average post type: {best_type}")
    print(f"  Highest-average platform: {best_platform}")

# Missing values and duplicate rows
print("\n4. Data Quality")
print("Missing values:")
display(df_filtered.isnull().sum().to_frame('Missing Values'))
print("Duplicate rows:", df_filtered.duplicated().sum())
print("Total posts analyzed:", len(df_filtered))
```

```
1. Average Engagement by Platform
```

platform	likes	comments	shares
Facebook	2669.28	248.91	474.38
Instagram	2993.89	232.44	525.17

```
2. Average Engagement by Post Type
```

post_type	likes	comments	shares
carousel	2352.68	276.89	602.95
image	2630.64	234.00	461.36
poll	3734.78	241.56	568.00
text	2197.85	158.62	336.85
video	3585.88	266.38	504.00

```
3. Highest Average Engagement Categories

Likes:
  Highest-average post type: poll
  Highest-average platform: Instagram

Comments:
  Highest-average post type: carousel
  Highest-average platform: Facebook

Shares:
  Highest-average post type: carousel
  Highest-average platform: Instagram

4. Data Quality
Missing values:
```

	Missing Values
post_id	0
platform	0
post_type	0
post_time	0
likes	0
comments	0
shares	0
post_day	0
sentiment_score	0
post_hour	0

```
Duplicate rows: 0
Total posts analyzed: 68
```

Check for Outliers

Potential outliers in likes, comments, and shares were examined using the Interquartile Range (IQR) method. No outliers were identified using this method, so all observations were retained.

In [27]:
```
# Identify potential outliers using the IQR method

engagement_columns = ['likes', 'comments', 'shares']

outlier_summary = []

for column in engagement_columns:

    Q1 = df_filtered[column].quantile(0.25)
    Q3 = df_filtered[column].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df_filtered[
        (df_filtered[column] < lower_bound) |
        (df_filtered[column] > upper_bound)
    ]

    outlier_summary.append({
        'Metric': column,
        'Lower Bound': round(lower_bound, 2),
```

```
        'Upper Bound': round(upper_bound, 2),
        'Number of Outliers': len(outliers)
    })

# Display outlier summary
outlier_summary_df = pd.DataFrame(outlier_summary)

display(outlier_summary_df)
```

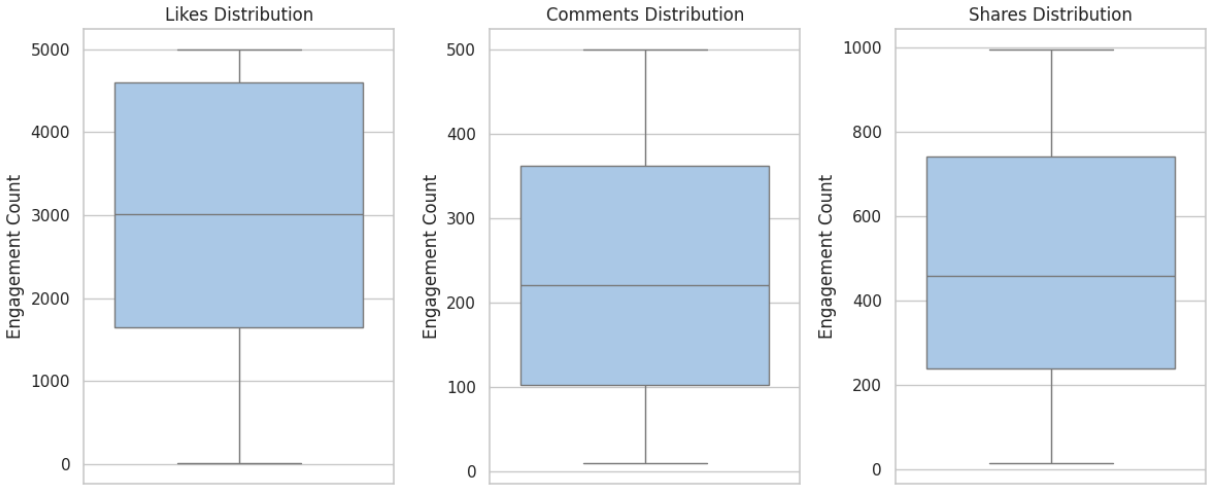
	Metric	Lower Bound	Upper Bound	Number of Outliers
0	likes	-2767.50	9026.50	0
1	comments	-284.88	750.12	0
2	shares	-511.00	1491.00	0

In [28]:
```
# Visualize potential outliers in engagement metrics

plt.figure(figsize=(12, 5))

for i, column in enumerate(engagement_columns, 1):
    plt.subplot(1, 3, i)
    sns.boxplot(y=df_filtered[column])
    plt.title(f'{column.capitalize()} Distribution')
    plt.ylabel('Engagement Count')

plt.tight_layout()
plt.show()
```



Feature Identification and Data Types

In [29]:
```
# Examine variables and their data types

print("Dataset columns and data types:")
display(df_filtered.dtypes)

print("\nUnique values in categorical variables:")
```

```
categorical_columns = ['platform', 'post_type', 'post_day']

for column in categorical_columns:
    print(f"\n{column}:")
    print(df_filtered[column].unique())
```

```
Dataset columns and data types:
```

	0
post_id	int64
platform	object
post_type	object
post_time	object
likes	int64
comments	int64
shares	int64
post_day	object
sentiment_score	object
post_hour	int32

dtype: object

```
Unique values in categorical variables:

platform:
['Facebook' 'Instagram']

post_type:
['image' 'carousel' 'poll' 'text' 'video']

post_day:
['Thursday' 'Sunday' 'Tuesday' 'Friday' 'Saturday' 'Monday' 'Wednesday']
```

Data Transformation: Total Engagement

A new target variable, total_engagement, was also created by adding likes, comments, and shares for each post. This provided a combined measure of engagement for the predictive modelling stage.

In [30]:
```
# Calculate total engagement for each post

df_filtered['total_engagement'] = (
    df_filtered['likes'] +
    df_filtered['comments'] +
    df_filtered['shares']
```

```
)

# Display the results
display(
    df_filtered[
        ['platform', 'post_type', 'post_day',
         'post_hour', 'likes', 'comments',
         'shares', 'total_engagement']
    ].head(10)
)
```

	platform	post_type	post_day	post_hour	likes	comments	shares	total_engagement
0	Facebook	image	Thursday	14	2121	474	628	3223
1	Facebook	carousel	Sunday	0	3660	432	694	4786
2	Instagram	poll	Tuesday	16	4955	408	688	6051
5	Instagram	carousel	Friday	20	256	186	211	653
6	Instagram	image	Sunday	11	1982	30	906	2918
7	Instagram	text	Saturday	19	1274	45	216	1535
8	Facebook	video	Tuesday	6	317	249	221	787
12	Facebook	carousel	Saturday	15	45	78	375	498
14	Facebook	carousel	Monday	20	889	314	262	1465
16	Instagram	poll	Wednesday	5	2872	10	736	3618

Selection of Independent and Dependent Variables

In [31]:
```
# Define independent variables (X)

X = df_filtered[
    ['platform', 'post_type', 'post_day', 'post_hour']
]

# Define dependent variable (Y)

y = df_filtered['total_engagement']

# Check the dimensions of X and Y

print("Independent variables (X):", X.shape)
print("Dependent variable (Y):", y.shape)

display(X.head())
display(y.head())
```

```
Independent variables (X): (68, 4)
Dependent variable (Y): (68,)
```

	platform	post_type	post_day	post_hour
0	Facebook	image	Thursday	14
1	Facebook	carousel	Sunday	0
2	Instagram	poll	Tuesday	16
5	Instagram	carousel	Friday	20
6	Instagram	image	Sunday	11

	total_engagement
0	3223
1	4786
2	6051
5	653
6	2918

dtype: int64

Splitting Data into Training and Testing Sets

The dataset was divided into training and testing subsets using an 80/20 split. This resulted in 54 observations for training and 14 observations for testing, out of the 68 total observations.

The train_test_split function from Scikit-learn was used with a random state of 42 to ensure reproducibility. A random split was selected because the available dataset contains posting days and hours but does not provide a complete chronological timestamp for establishing a reliable time-based split.

Time-Based Validation and Shuffling

A random 80/20 train-test split was considered appropriate because this study focuses on identifying general engagement patterns rather than forecasting future engagement. The dataset does not contain complete chronological timestamps, making a time-based split impractical. Random splitting allows the model to learn from 54 observations and evaluate its predictions on 14 unseen observations. A fixed random state of 42 ensures

reproducibility. However, the results should not be interpreted as evidence of future predictive performance.

In [32]:
```
from sklearn.model_selection import train_test_split

# Split the original, unencoded features

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training features:", X_train.shape)
print("Testing features:", X_test.shape)
print("Training target:", y_train.shape)
print("Testing target:", y_test.shape)
```

```
Training features: (54, 4)
Testing features: (14, 4)
Training target: (54,)
Testing target: (14,)
```

Baseline Model Development: Linear Regression

Preprocessing categorical data with OneHotEncoder

The dataset contained three categorical predictor variables: platform, post type, and day of the week. These variables were transformed using One-Hot Encoding through Scikit-learn's OneHotEncoder.

The drop='first' parameter was applied to reduce redundant categories and avoid multicollinearity associated with including all category indicators in the Linear Regression model. The handle_unknown='ignore' parameter was also used to accommodate any previously unseen categories during prediction.

Preprocessing Numerical Data

The numerical variable, post_hour, was passed directly into the model using Scikit-learn's ColumnTransformer. No Min-Max scaling or standardization was applied to the numerical predictor variable. Since this was the only numerical input feature and its values were already represented on a consistent 24-hour scale (0–23), it was retained in its original format.

Leakage Prevention

The preprocessing steps were integrated into the Linear Regression pipeline, ensuring that the encoder was fitted on the training data only and subsequently applied to the testing data. This helped prevent data leakage and maintained consistency throughout model development and evaluation.

In [33]:
```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression

# Identify categorical and numerical features

categorical_features = ['platform', 'post_type', 'post_day']
numerical_features = ['post_hour']

# Create preprocessing step

preprocessor = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(drop='first',
                              handle_unknown='ignore'), categorical_features),
        ('num', 'passthrough', numerical_features)
    ]
)

# Create Linear Regression pipeline

lr_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

# Train the model

lr_model.fit(X_train, y_train)

# Generate predictions

y_pred_lr = lr_model.predict(X_test)

# Display actual and predicted engagement

lr_results = pd.DataFrame({
    'Actual Engagement': y_test.values,
    'Predicted Engagement': y_pred_lr.round(2)
})

display(lr_results)
```

	Actual Engagement	Predicted Engagement
0	6120	5370.34
1	5549	3540.00
2	2918	4439.19
3	3618	4746.37
4	5191	3978.91
5	1723	4246.47
6	3715	1914.49
7	1535	2920.81
8	2417	3431.30
9	2577	2416.12
10	3486	2915.32
11	1241	2657.42
12	5669	3245.64
13	3223	2546.30

Baseline Model Evaluation: Linear Regression

Interpretation of Baseline Model Results

The Linear Regression model achieved an MAE of 1,328.03 and an R-squared value of 0.0706. This indicates limited predictive performance, with the model explaining approximately 7.06% of the variation in engagement within the test set. The results suggest that platform, post type, posting day and posting hour alone do not adequately explain engagement levels. The small dataset and test sample may also affect the reliability of these results.

In [34]:
```
from sklearn.metrics import mean_absolute_error, r2_score

# Evaluate the corrected Linear Regression model

mae_lr = mean_absolute_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

print("Linear Regression Model Performance")
print("-----------------------------------")
print("Mean Absolute Error (MAE):", round(mae_lr, 2))
print("R-squared (R2):", round(r2_lr, 4))
```

```
Linear Regression Model Performance
-----------------------------------
Mean Absolute Error (MAE): 1328.03
R-squared (R2): 0.0706
```

In [35]:
```python
# Create a performance metrics table

baseline_metrics = pd.DataFrame({
    'Evaluation Metric': ['Mean Absolute Error (MAE)', 'R-squared (R²)'],
    'Result': [round(mae_lr, 2), round(r2_lr, 4)]
})

display(baseline_metrics)
```

	Evaluation Metric	Result
0	Mean Absolute Error (MAE)	1328.0300
1	R-squared (R^2)	0.0706